

Lab Task 1: Bank Account Management System (Basic)

Objective:

Create a basic bank account system with the ability to perform deposits, withdrawals, and maintain a transaction history. The task will help you understand object-oriented programming concepts such as classes, properties, and methods in C#.

Instructions:

1. Create a BankAccount Class:

- **Static Property:**

- **BankName:** A static property representing the bank's name. It should be initialized to "Global Trust Bank" using a static constructor.

- **Instance Properties:**

- **AccountName:(String)** The name of the account holder (read/write).
- **AccountNumber:(long)** The unique account number (read-only).
- **Balance:(decimal)** The account balance (protected and modifiable only within the class).
- **AccountType:(Enum)** Type of account (e.g., "Current, Saving, Bussiness") (read/write, default value: "Generic").
- **IsActive:(Boolean)** The account's status (read-only, default value true).

- TransactionHistory: A list of strings that stores the transaction history (read-only outside the class).

2. Constructor:

- Accepts accountName and accountNumber as parameters.
- Initialize the balance to 0.0 and add a log entry for account creation to the TransactionHistory.

3. Methods:

- Deposit(double amount): Increases the balance and logs the deposit in the transaction history. Ensure the deposit amount is positive.
- Withdrawal(double amount): Decreases the balance and logs the withdrawal. Ensure the withdrawal amount is positive and there are sufficient funds.
- DisplayAccountInfo(): Displays all the account details, including the balance and transaction history.

4. Properties:

- For properties that should be protected or read-only, you will define getter and setter logic.
- AccountName: Should be get and set by default.
- AccountNumber: Should be a read-only property with a get method only.
- Balance: Should be a protected property, and you can define methods to modify it internally.
- TransactionHistory: This list should only be accessed in a read-only manner outside the class

4. **Test the Class:**

- Create an instance of BankAccount in your Main method and perform the following actions:
 - Deposit some money into the account.
 - Make a withdrawal from the account.
 - Display account information including the transaction history.

Lab Task 2: Bank Account Management System with Inheritance and Polymorphism

Objective:

In this task, you will extend the concepts from Lab Task 2, which focused on creating a basic bank account system. The goal now is to enhance the system by adding multiple types of bank accounts (e.g., Savings, Checking) using Inheritance and Polymorphism. This will give you a deeper understanding of object-oriented programming (OOP) concepts in C#, such as class inheritance, method overriding, and polymorphic behavior.

Instructions:

1. Extend the BankAccount Class:

- You should use the BankAccount class from **Lab 2** as the base class.
- Create two new derived classes: SavingsAccount and CheckingAccount.

2. Derived Classes:

- **SavingsAccount Class:**
 - **Instance Properties:**
 1. Intrest Rate = 4%[Const]
 - This class should inherit from BankAccount.
 - It should override the DisplayAccountInfo() method to show additional information, like the interest rate.
 - It should have a method to apply interest to the balance (ApplyInterest()). Use base.Desposit() to deposit intrest amount

3. Test the Bank Accounts:

- Create instances of SavingsAccount in your Main method and perform operations like deposits, withdrawals, ApplyInterest, and displaying account information.
- Use a **base class reference** to call overridden methods of the derived classes.

